

Documento Técnico: Pruebas de Despliegue Separado (Frontend y Backend) — Proyecto HeartCoin

Introducción

Este documento examina cómo se prueban, de forma independiente, el cliente Flutter (frontend) y el backend Supabase (base de datos, autenticación, funciones y tiempo real) del proyecto HeartCoin, antes y después de cualquier despliegue. Cubre tanto la infraestructura de pruebas automatizadas realmente presente en el repositorio como la verificación manual efectivamente documentada, y detalla, sin omitir información, el estado de cada mecanismo: qué existe, qué se ejecutó, con qué resultado exacto, y qué continúa dependiendo por completo de verificación manual.

Objetivo

Determinar con precisión qué mecanismos de prueba existen hoy para el frontend y para el backend de HeartCoin de forma separada, confirmar su resultado real de ejecución, y documentar la verificación manual que complementa —o en algunos casos sustituye— a la automatización, de modo que cualquier persona que retome el proyecto conozca exactamente qué garantías de calidad existen antes de considerar un cambio listo para despliegue.

Alcance

Este documento cubre las pruebas del cliente Flutter (carpeta `test/`, análisis estático, builds de verificación) y del backend Supabase (Edge Functions, migraciones SQL, separación de entornos), así como la superficie de prueba de las dos integraciones externas de mayor relevancia transaccional (Original Auth y Original Pay). No cubre pruebas de carga, pruebas de penetración, ni auditoría de seguridad formal, por no existir ningún artefacto de ese tipo en el repositorio al momento de este análisis.

Metodología

La información se obtuvo mediante inspección directa de la carpeta `test/`, la carpeta `supabase/` (incluyendo Edge Functions, sus helpers compartidos y metadata del CLI), `SQL/migrations/`, `analysis_options.yaml` y `pubspec.yaml`, contrastada contra la ejecución real de `flutter test`,

`flutter analyze` y `deno test` sobre el estado actual del repositorio —incluyendo las suites de pruebas de frontend y de backend incorporadas en esta misma ronda de trabajo—. Se revisaron además todos los documentos `.md` de la raíz del proyecto en busca de menciones explícitas a pruebas, verificación o QA, citando el texto literal encontrado en cada caso para evitar interpretaciones no respaldadas por el propio texto.

Desarrollo del Análisis

1. Pruebas Automatizadas del Frontend

La carpeta `test/` del proyecto contiene un único archivo: `test/widget_test.dart`. Hasta antes de esta ronda de trabajo, ese archivo era la plantilla sin modificar que genera automáticamente el comando `flutter create` (el "smoke test del contador"), y fallaba al ejecutarse porque asumía una pantalla de ejemplo que nunca existió en HeartCoin. Ese archivo fue reemplazado por una suite real de ocho pruebas, agrupadas en cuatro conjuntos, cada una verificando comportamiento efectivo de pantallas y componentes propios de la aplicación:

Conjunto	Prueba	Qué verifica
OnboardingScreen	Muestra la primera página con su título y el botón "Siguiente"	El texto "Bienvenid@ a HeartCoin" y el botón "Siguiente" están presentes al montar la pantalla; el botón "Empezar" no lo está todavía.
	El botón cambia a "Empezar" y navega a HomeScreen al llegar a la última página	Tras avanzar las tres páginas del recorrido, el texto de la última página aparece, el botón cambia su etiqueta, y al presionarlo la navegación reemplaza la pantalla por HomeScreen (verificado por tipo de widget, no solo por texto).
HomeScreen	El botón "Registrarse" navega a RegisterScreen	La pantalla de bienvenida enlaza correctamente al formulario de registro.
	El botón "Iniciar sesión" navega a LoginScreen	La pantalla de bienvenida enlaza correctamente al formulario de inicio de sesión.
LoginScreen	Muestra un error y no intenta iniciar sesión si el correo y la contraseña están vacíos	La validación de campos vacíos se dispara correctamente y nunca llega a invocar el servicio de autenticación real.
	El enlace "¿Olvidaste tu contraseña?" está presente	El punto de entrada al flujo de recuperación de contraseña existe en la pantalla.

Conjunto	Prueba	Qué verifica
AppChip	Refleja el estado seleccionado en su color de fondo	El widget de chip compartido (usado en Explorar y en los filtros de Iniciativas de ahorro) cambia su color de fondo según la propiedad <code>selected</code> .
	Dispara <code>onTap</code> al presionarlo	El callback de interacción del chip se ejecuta correctamente al tocarlo.

Se ejecutó `flutter test` sobre el estado actual del repositorio para confirmar el resultado real de esta suite:

```
00:00 +0: OnboardingScreen muestra la primera página con su título y el botón "Siguiente"
00:00 +1: OnboardingScreen el botón cambia a "Empezar" y navega a HomeScreen al llegar a la última página
00:01 +2: HomeScreen el botón "Registrarse" navega a RegisterScreen
00:01 +3: HomeScreen el botón "Iniciar sesión" navega a LoginScreen
00:01 +4: LoginScreen muestra un error y no intenta iniciar sesión si el correo y la contraseña están vacíos
00:01 +5: LoginScreen el enlace "¿Olvidaste tu contraseña?" está presente
00:01 +6: AppChip refleja el estado seleccionado en su color de fondo
00:01 +7: AppChip dispara onTap al presionarlo
00:01 +8: All tests passed!
```

Las ocho pruebas pasan. Esta es la primera vez que el proyecto cuenta con pruebas automatizadas de frontend con resultado exitoso verificado; hasta antes de esta ronda, el único archivo de prueba existente fallaba al ejecutarse.

Diseño de aislamiento: por qué estas pruebas no dependen de red ni de Supabase

Una decisión deliberada de diseño de esta suite es que ninguna de sus ocho pruebas requiere que Supabase esté inicializado ni que exista conectividad de red, lo cual las hace ejecutables en cualquier entorno (incluida una futura integración continua) sin necesidad de credenciales ni de un backend disponible. Esto se logra de dos formas distintas según el caso:

- **Pantallas que no invocan Supabase antes del punto verificado:** `OnboardingScreen` y `HomeScreen` no tocan ningún servicio de backend en su construcción ni en la navegación entre ellas — son pantallas puramente de presentación y navegación local.
- **Interrupción de la ejecución antes de la llamada de red:** en el caso de `LoginScreen`, el método `_onSignIn()` valida primero, de forma síncrona, que el correo y la contraseña no estén vacíos, y solo si esa validación pasa procede a invocar `AuthService.instance.signIn()` (que sí requiere

Supabase inicializado). La prueba envía el formulario vacío deliberadamente, de modo que la ejecución nunca llega a ese punto y el resultado observado (el mensaje de error) es enteramente determinista.

Cada prueba envuelve la pantalla bajo análisis en un `MaterialApp` mínimo propio de la prueba (no la aplicación completa `MyApp`), evitando así la inicialización real de Supabase que solo ocurre dentro de la función `main()` del proyecto y que sería necesaria para poder instanciar `MyApp` de forma completa en un entorno de prueba.

Problemas encontrados y corregidos durante la construcción de la suite

La elaboración de estas pruebas no fue trivial en dos puntos concretos, documentados aquí porque son relevantes para cualquier prueba futura sobre las mismas pantallas:

- **Texto ambiguo en LoginScreen:** la cadena "Iniciar sesión" aparece dos veces en esa pantalla — como título grande de la pantalla y como etiqueta del botón de envío. Un primer intento de localizar el botón mediante `find.text('Iniciar sesión')` falló porque el buscador de Flutter Test encontró ambas coincidencias y no pudo determinar cuál accionar. Se resolvió acotando la búsqueda al tipo de widget específico (`find.widgetWithText(ElevatedButton, 'Iniciar sesión')`), que sí identifica de forma única el botón.
- **Botón fuera del área visible de la prueba:** el entorno de prueba de widgets de Flutter renderiza, por defecto, un lienzo de 800×600 píxeles lógicos. El botón de inicio de sesión de `LoginScreen` vive dentro de una tarjeta desplazable de altura fija (la mitad de la pantalla) con un formulario más largo que ese espacio visible en la resolución de prueba, quedando su posición calculada fuera de los límites del lienzo — el intento de tocarlo directamente generó una advertencia explícita de la propia herramienta de pruebas ("derived an Offset that would not hit test on the specified widget"). Se resolvió invocando `tester.ensureVisible(...)` antes de la interacción, que desplaza automáticamente el contenido hasta que el widget objetivo es alcanzable, replicando lo que un usuario real haría al deslizar la pantalla.

Análisis estático

Complementario a las pruebas funcionales, el análisis estático de código sigue configurado en `analysis_options.yaml` sobre el conjunto de reglas recomendado por el propio equipo de Flutter (`package:flutter_lints/flutter.yaml`), sin reglas adicionales propias del proyecto activadas. Se ejecutó `flutter analyze` sobre el estado actual del repositorio, incluyendo el nuevo archivo de pruebas:

```
Analyzing heartcoin...
  info - 'anonKey' is deprecated ... - lib/main.dart:34:5
warning - ... unused_element_parameter - lib/screens/auth/register_screen.dart:712:10
warning - ... unused_field (x7) - lib/screens/onboarding/onboarding_screen.dart
```

12 issues found.

El resultado es idéntico al observado antes de incorporar la nueva suite de pruebas: cero errores de compilación o de tipo, un aviso informativo de una API en vías de discontinuación, y once advertencias de estilo menor (parámetros y campos declarados pero no utilizados) preexistentes en pantallas no relacionadas con las pruebas nuevas. El archivo `test/widget_test.dart` en su versión actual no introduce ningún nuevo hallazgo de análisis estático.

Alcance actual de la cobertura y lo que aún queda sin cubrir

Precisión sobre el alcance real: las ocho pruebas incorporadas cubren el flujo de entrada de la aplicación (onboarding, bienvenida, navegación inicial de autenticación, validación básica de formulario) y un componente visual compartido. **No cubren** —y esto debe entenderse con claridad antes de considerar el frontend "probado" en un sentido amplio— los flujos posteriores al inicio de sesión (feed, Wallet, canje de HeartCoin, votaciones, creación de iniciativas o beneficios, ni ninguna de las pantallas específicas de los roles Organización o Empresa), ni tampoco el camino exitoso de inicio de sesión o registro, que requeriría simular respuestas de Supabase (mediante un cliente falso o de prueba) en lugar de depender de la red real. Es una base real y funcional sobre la que construir, no una cobertura integral del sistema.

Builds de verificación documentados

`MANUAL_TECNICO.md` documenta la generación exitosa del paquete de distribución de release (`app-release.aab` , 72.6 MB, firmado y verificado), descrita con mayor detalle en el documento técnico de despliegue independiente de la aplicación. No se encontró, en ningún documento de la raíz del proyecto, mención de un build de verificación en modalidad debug ejecutado como parte de un proceso repetible — la única verificación de compilación documentada corresponde al momento de generar el paquete final de release.

2. Pruebas Automatizadas del Backend

El backend de HeartCoin es Supabase (Postgres administrado, Auth, Realtime, Storage y Edge Functions), complementado por siete funciones Edge propias del proyecto:

```
supabase/functions/delete-account/index.ts
supabase/functions/originalauth-register/index.ts
supabase/functions/originalauth-resend-verification/index.ts
supabase/functions/originalauth-verify-email/index.ts
supabase/functions/originalpay-balance/index.ts
supabase/functions/originalpay-redeem/index.ts
```

```
supabase/functions/originalpay-webhook/index.ts
supabase/functions/upload-media/index.ts
```

Hasta antes de esta ronda de trabajo, no existía en el repositorio ningún archivo de prueba para estas funciones, ni evidencia de un ejecutor de pruebas de Deno instalado en el entorno de desarrollo. En esta ronda se instaló el CLI de Deno y se incorporaron dos archivos de prueba dirigidos a los dos módulos compartidos (`_shared/`) que concentran la lógica de mayor sensibilidad de seguridad del sistema: la firma y verificación de peticiones HTTP entre HeartCoin y sus dos integraciones externas.

Pruebas del cliente firmado hacia Original Auth

`supabase/functions/_shared/originalauth.test.ts` incorpora ocho pruebas sobre `_shared/originalauth.ts`, el módulo que construye y firma las peticiones salientes hacia la API de Original Auth (registro, verificación de correo, reenvío de código):

- **Cinco pruebas sobre `profileTableForRole`** (función pura de mapeo de rol a tabla de perfil): cada uno de los tres roles válidos (`personal`, `organizacion`, `empresa`) resuelve a su tabla correspondiente, y tanto un rol no reconocido como un rol indefinido caen correctamente al valor por defecto (`personal_profiles`).
- **Tres pruebas sobre `originalAuthRequest`**, verificadas contra un servidor HTTP local levantado dentro del propio archivo de prueba (no contra la red real de Original Auth): confirman que cada petición incluye un identificador de un solo uso (*nonce*) distinto incluso para el mismo cuerpo de solicitud, que dos cuerpos de solicitud distintos producen firmas HMAC distintas, y que el código de estado HTTP y el cuerpo de la respuesta del servidor se propagan sin alteración hacia quien invoca la función.

Pruebas del verificador de firma entrante de Original Pay

`supabase/functions/_shared/originalpay_auth.test.ts` incorpora once pruebas sobre `_shared/originalpay_auth.ts`, el módulo que verifica la autenticidad de cada petición que Original Pay envía hacia HeartCoin — la superficie de seguridad más sensible del sistema, dado que protege los tres endpoints que mueven HeartCoin real:

- **Cuatro pruebas sobre `canonicalPathWithQuery`**: construcción correcta de la ruta canónica sin parámetros de consulta, ordenamiento alfabético determinista de los parámetros sin importar el orden de llegada, y preservación de valores repetidos de un mismo parámetro.
- **Siete pruebas sobre `verifyOriginalPaySignature`**, cada una construyendo una petición HTTP firmada de la misma forma en que lo haría Original Pay: acepta una firma válida construida con la clave compartida correcta; rechaza una clave de API incorrecta; rechaza una petición sin el header de marca de tiempo; rechaza una marca de tiempo fuera de la ventana de ± 5 minutos que protege contra ataques de repetición (*replay*); rechaza una firma que no corresponde al cuerpo exacto de

la petición (detectando manipulación del contenido después de firmado); rechaza una firma calculada para una ruta distinta a la verificada (detectando confusión de ruta); y rechaza una firma con formato inválido.

Se ejecutó `deno test` sobre ambos archivos, en conjunto, para confirmar el resultado real:

```
running 8 tests from ./supabase/functions/_shared/originalauth.test.ts
profileTableForRole: personal ... ok
profileTableForRole: organizacion ... ok
profileTableForRole: empresa ... ok
profileTableForRole: un rol no reconocido cae al perfil personal (valor por defecto) ... ok
profileTableForRole: rol indefinido cae al perfil personal (valor por defecto) ... ok
originalAuthRequest: firma cada petición con headers distintos (nonce único) ... ok
originalAuthRequest: dos payloads distintos producen firmas distintas ... ok
originalAuthRequest: propaga el status HTTP y el cuerpo de la respuesta ... ok
running 11 tests from ./supabase/functions/_shared/originalpay_auth.test.ts
canonicalPathWithQuery: sin query params devuelve solo el path ... ok
canonicalPathWithQuery: ordena los parámetros alfabéticamente ... ok
canonicalPathWithQuery: mismo resultado sin importar el orden original ... ok
canonicalPathWithQuery: conserva valores repetidos de un mismo parámetro ... ok
verifyOriginalPaySignature: acepta una firma válida ... ok
verifyOriginalPaySignature: rechaza una API key incorrecta ... ok
verifyOriginalPaySignature: rechaza si falta el header de timestamp ... ok
verifyOriginalPaySignature: rechaza timestamp fuera de la ventana anti-replay ... ok
verifyOriginalPaySignature: rechaza body manipulado ... ok
verifyOriginalPaySignature: rechaza path distinto al firmado ... ok
verifyOriginalPaySignature: rechaza firma con formato inválido ... ok

ok | 19 passed | 0 failed
```

Las diecinueve pruebas pasan. Es la primera cobertura automatizada que existe sobre el backend de HeartCoin, y se concentra deliberadamente en la superficie de mayor riesgo: la autenticación entre servicios que protege el movimiento real de HeartCoin.

Un detalle técnico de Deno documentado durante la construcción de estas pruebas

Ambos módulos bajo prueba leen sus variables de entorno (`ORIGINAL_AUTH_BASE_URL`, `ORIGINAL_AUTH_API_KEY`, `ORIGINALPAY_API_KEY`) a nivel de archivo, fuera de cualquier función — un patrón común en Deno, pero que tiene una consecuencia poco intuitiva para quien escribe pruebas: **fijar esas variables con `Deno.env.set(...)` dentro del propio archivo de prueba no funciona**, porque los `import` estáticos de JavaScript/TypeScript se resuelven antes de ejecutar cualquier código de nivel superior que los siga en el archivo — el módulo bajo prueba se importa, y por tanto lee sus variables de entorno, antes de que la línea que las fija llegue a ejecutarse. La solución

correcta, aplicada y verificada en ambos archivos, es exportar esas variables en la sesión de terminal **antes** de invocar `deno test` :

```
ORIGINAL_AUTH_BASE_URL=http://127.0.0.1:8971 \
ORIGINAL_AUTH_API_KEY=clave-de-prueba-original-auth \
ORIGINALPAY_API_KEY=clave-de-prueba-original-pay \
deno test --allow-env --allow-net supabase/functions/_shared/
```

Este comportamiento queda documentado explícitamente como comentario al inicio de ambos archivos de prueba, para que no vuelva a costar tiempo de diagnóstico a quien los ejecute o los extienda en el futuro.

Diseño de aislamiento de las pruebas de backend

Igual que en el frontend, ninguna de las diecinueve pruebas de backend depende de la red real de Original Auth o de Original Pay, ni de credenciales reales de esos servicios: las pruebas sobre `originalAuthRequest` levantan un servidor HTTP local efímero dentro del propio proceso de prueba (en un puerto de la máquina local, cerrado explícitamente al finalizar cada prueba), y las pruebas sobre `verifyOriginalPaySignature` construyen las peticiones HTTP firmadas directamente en memoria, sin red alguna. Esto permite que la suite se ejecute en cualquier entorno —incluida una futura integración continua— sin depender de que los servicios externos estén disponibles ni de exponer credenciales reales en un pipeline automatizado.

No se modificó ningún archivo de producción para lograr esta cobertura. Ambas Edge Functions y sus siete archivos `index.ts` permanecen exactamente como estaban; solo se agregaron los dos archivos de prueba nuevos. Esto fue una decisión deliberada: la lógica de negocio propia de cada una de las siete funciones (validación de campos del cuerpo de la solicitud, parseo del nombre completo del usuario, mapeo de errores de negocio a mensajes de respuesta) vive hoy inline dentro del manejador `Deno.serve(...)` de cada función, no en funciones exportadas de forma independiente — por lo que, a diferencia de los dos módulos compartidos ya cubiertos, esa lógica **no es testeable de forma aislada sin antes refactorizarla** para extraerla a funciones nombradas y exportadas, tal como ya ocurre con `_shared/originalauth.ts` y `_shared/originalpay_auth.ts` . Esa extracción no se realizó en esta ronda por tratarse de una modificación a código ya desplegado en producción, que amerita su propia revisión y decisión explícita antes de emprenderse.

De igual forma, no existe en `SQL/` ningún archivo con convención de nombre de prueba (`test_*.sql`) ni evidencia de un framework de pruebas para Postgres (como pgTAP): las funciones RPC de mayor relevancia transaccional (`redeem_beneficio` , `redeem_servicio` , `checkin_iniciativa`)

y las políticas de seguridad a nivel de fila (RLS) del proyecto continúan sin ninguna cobertura automatizada, dado que requieren una instancia real de Postgres —local o en la nube— contra la cual ejecutarse, y esa pieza de infraestructura de pruebas está fuera del alcance abordado en esta ronda.

Migraciones de esquema versionadas

Sí existe, en cambio, un mecanismo de control de cambios sobre el esquema de base de datos: seis archivos de migración versionados numéricamente en `SQL/migrations/`:

```
001_iniciativa_participantes.sql
002_beneficios_servicios_ubicacion.sql
003_publicaciones_iniciativa.sql
004_beneficios_destacado.sql
005_organization_profiles_lectura_publica.sql
006_servicios_destacado.sql
```

`MANUAL_TECNICO.md` confirma explícitamente que este control de versiones no existía antes: "no existían migraciones versionadas antes de esta ronda; el esquema se aplicaba directo en Supabase Studio". Cada archivo se aplica de forma manual, uno a la vez, mediante el comando `supabase db query --linked -f <archivo>`, sin ningún script auxiliar (`Makefile`, script de shell o PowerShell, tarea de `package.json`) que automatice su aplicación en orden ni que registre cuáles ya se ejecutaron contra el proyecto activo — el único control de versión es el prefijo numérico del nombre de archivo, interpretado manualmente por quien aplica el cambio.

Ausencia de configuración local reproducible

El repositorio no contiene un archivo `supabase/config.toml`, que es el artefacto que permitiría levantar una instancia local completa de Supabase (`supabase start`) para desarrollo o pruebas aisladas de la base de datos en la nube. Lo que sí existe es metadata generada automáticamente por el CLI al enlazar el proyecto remoto (`supabase/.temp/linked-project.json`, con referencia al proyecto `bamrvwzpcqwoyuwbvomw`), confirmando que el flujo de trabajo real del equipo es enlazar directo contra el proyecto de Supabase en la nube y operar siempre contra él —incluyendo, según lo documentado, la aplicación de migraciones— sin una instancia local intermedia. Esta ausencia es, además, la razón concreta por la que las funciones RPC y las políticas de RLS no pudieron cubrirse en esta misma ronda de trabajo: levantar `supabase start` requeriría primero crear este archivo de configuración y decidir cómo poblar una base de datos de prueba con el esquema actual.

3. Separación de Entornos

Existe un único proyecto de Supabase para todo el ciclo de vida del sistema: `bamrvwzpcqwoyuwbvomw` (nombrado "heartcoin-v1" en el panel de Supabase), referenciado de forma idéntica en `lib/main.dart`, en la documentación de endpoints y en la de integraciones externas.

`MANUAL_TECNICO.md` menciona la existencia de un proyecto anterior ("heartcoin", inactivo), pero corresponde a un proyecto abandonado, no a un entorno de staging paralelo en uso.

Esto significa que **no existe, dentro del propio HeartCoin, ningún ambiente de pruebas separado del ambiente de producción**: toda prueba manual realizada durante el desarrollo —incluidas las cuentas de demostración documentadas en `CUENTAS_DEMO.md` — se ejecuta contra la misma base de datos que sirve a cualquier usuario real de la aplicación. Las nuevas pruebas automatizadas de frontend y de backend descritas en las secciones 1 y 2, en cambio, están deliberadamente diseñadas para no depender de ese proyecto ni de ningún otro: no tocan Supabase, ni la red real de Original Auth u Original Pay, precisamente para no verse afectadas por esta limitación.

Matiz importante: sí existe una noción de "ambiente de prueba" (`test` vs. `prod`) en el repositorio, pero pertenece exclusivamente al servicio externo **Original Auth**, no a HeartCoin ni a Supabase. Ese servicio distingue explícitamente su propio ambiente de pruebas (`api.test.originalauth.com`) del de producción, con comportamiento distinto documentado en `README.md` (por ejemplo, el ambiente `test` de Original Auth no envía correos reales y en su lugar expone el código de verificación directamente en la respuesta de la API). Esta distinción se filtró de forma visible hasta la interfaz de la aplicación: `VerifyEmailScreen` muestra un aviso explícito en pantalla — "Ambiente de prueba: el código se autocompletó porque este correo no se envía de verdad en test." — cuando la respuesta del backend indica que se está operando contra ese ambiente. Es, en la práctica, el único mecanismo de separación de entornos verificable contra los servicios reales en todo el sistema, y no cubre a Supabase ni a la base de datos propia de HeartCoin.

4. Verificación Manual Documentada

Más allá de las pruebas automatizadas ya incorporadas en frontend y backend, la verificación del resto del sistema —en particular, las funciones RPC transaccionales y el comportamiento de extremo a extremo de las integraciones externas— queda registrada de forma dispersa en varios documentos narrativos, no como un plan de pruebas formal. Se citan a continuación los pasajes más relevantes, tal como aparecen en el repositorio:

Documento	Cita literal relevante
<code>MANUAL_TECNICO.md</code> , §9	"Se generó exitosamente <code>app-release.aab</code> (72.6 MB), firmado y verificado." — y, sobre lo pendiente: "usar las cuentas demo de <code>CUENTAS_DEMO.md</code> como credenciales de prueba para el revisor" de Google Play.

Documento	Cita literal relevante
CUENTAS_DEMO.md	Documenta cuentas de demostración por rol, todas creadas directamente mediante la API administrativa de Supabase (no a través del formulario de registro de la aplicación), con correo ya confirmado, más tres cuentas adicionales identificadas explícitamente como "cuentas reales de prueba usadas durante el desarrollo, no parte de la siembra".
ORIGINAL_PAY_INTEGRATION.md , línea 5	"Estado: implementado y probado. Los 3 endpoints (consulta de saldo, cobro, webhook de notificaciones) están desplegados y verificados de punta a punta, incluyendo el flujo completo de reembolso automático ante un pago rechazado."
ORIGINAL_PAY_INTEGRATION.md , línea 147	"Probado de punta a punta: cobro real → webhook rejected → HC devuelto correctamente → reintento del mismo webhook confirmado que no duplica el reembolso."
README.md , sección "Caveats descubiertos probando contra el ambiente test "	Documenta hallazgos concretos de pruebas manuales contra el ambiente de pruebas de Original Auth: un campo de formato incompatible detectado y corregido, exposición del código de verificación en modo prueba, y la necesidad de capturar explícitamente el tipo de excepción <code>FunctionException</code> del SDK de Supabase.

Ninguno de estos registros corresponde a una suite de pruebas ejecutable, un script reproducible, ni una checklist formal de QA: son, en todos los casos, relatos en prosa de una verificación puntual realizada una vez durante el desarrollo. La verificación de que la *lógica de firma HMAC en sí misma* es correcta ya cuenta, a partir de esta ronda, con pruebas automatizadas reproducibles (sección 2); lo que sigue dependiendo por completo de verificación manual es el comportamiento de extremo a extremo del sistema completo —incluyendo la respuesta real de los servidores de Original Auth y Original Pay, y las funciones RPC de la base de datos—, para lo cual no existe, en ningún documento del repositorio, un plan de pruebas, un runbook de despliegue, ni una checklist de verificación pre-publicación estructurada como tal.

5. Superficie de Prueba de las Integraciones Externas

Las dos integraciones externas de mayor relevancia transaccional —Original Auth (verificación de identidad) y Original Pay (movimiento de HeartCoin)— documentan su estado de prueba de forma desigual entre sí, lo cual constituye un hallazgo relevante por sí mismo.

Original Pay

`ORIGINAL_PAY_INTEGRATION.md` es, de los dos documentos, el que declara con mayor precisión un testing real ya ejecutado (citado en la tabla anterior). Documenta también, en su propia sección "Pendiente de definir", una condición de carrera conocida y no resuelta en el endpoint de cobro (`redeem`): la ausencia de atomicidad entre la verificación de saldo suficiente y la inserción de la transacción correspondiente, reconocida explícitamente como riesgo abierto, sin ningún test de concurrencia que lo cubra. El webhook de notificaciones se autentica mediante un secreto compartido en el cuerpo de la solicitud —no mediante HMAC como los otros dos endpoints—, descrito en el propio documento como "más débil que HMAC (sin protección anti-replay real)", una decisión de diseño impuesta por Original Pay y no por HeartCoin. Las nuevas pruebas de la sección 2 cubren precisamente el mecanismo HMAC que protege a los otros dos endpoints (`balance` y `redeem`), pero no alcanzan al webhook, que usa un mecanismo de autenticación distinto y más simple.

Original Auth

Discrepancia documental detectada: el encabezado de `ORIGINAL_AUTH_INTEGRATION.md` indica literalmente "Estado: acordado con Nico, pendiente de implementación." Esta afirmación **no corresponde al estado real del código:** las tres Edge Functions de esta integración (`originalauth-register` , `originalauth-verify-email` , `originalauth-resend-verification`) ya están completamente implementadas y en uso, y el propio `README.md` documenta el flujo como operativo, incluyendo hallazgos de pruebas manuales ya realizadas contra el ambiente de prueba del servicio. Este documento quedó desactualizado respecto al avance real del proyecto y no debe tomarse como referencia del estado actual de esta integración.

La evidencia real de verificación de esta integración no vive en `ORIGINAL_AUTH_INTEGRATION.md` —que es, en cambio, un documento de alcance y diseño orientado a explicar a un interlocutor no técnico qué hace y qué no hace el servicio, con una sección de manejo de errores esperados pero sin plan de pruebas—, sino en los hallazgos exploratorios documentados en `README.md` , ya citados en la sección anterior, complementados ahora por las pruebas automatizadas del mecanismo de firma descritas en la sección 2.

Problemas Detectados

- 1. Lógica de negocio de las Edge Functions aún no testeable de forma aislada:** las siete funciones mantienen su validación de campos, parseo de datos y mapeo de errores de negocio inline dentro del manejador `Deno.serve(...)` , sin extraer a funciones exportadas — a diferencia de los dos módulos compartidos de firma, que sí quedaron cubiertos en esta ronda.
- 2. Ausencia total de pruebas automatizadas sobre las funciones RPC y las políticas de RLS de la base de datos** (`redeem_beneficio` , `redeem_servicio` , `checkin_iniciativa` , entre otras), dado

que requieren una instancia real de Postgres contra la cual ejecutarse y el proyecto no cuenta hoy con una instancia local reproducible (`supabase/config.toml` no existe).

3. **Cobertura de frontend aún acotada al flujo de entrada:** las ocho pruebas de frontend verifican onboarding, bienvenida, navegación inicial de autenticación y un componente visual compartido, pero no cubren ningún flujo posterior al inicio de sesión, ni el camino exitoso de login/registro contra un backend simulado.
4. **Ausencia de separación de entornos propia del proyecto:** un único proyecto de Supabase sirve simultáneamente como entorno de desarrollo, de prueba manual y de producción, exponiendo cualquier verificación manual a escribir directamente sobre datos reales.
5. **Aplicación manual y no automatizada de migraciones de esquema,** sin script que garantice el orden correcto ni registro de qué migraciones ya se aplicaron contra el proyecto activo.
6. **Verificación manual dispersa y no formalizada** para todo lo que queda fuera de las nuevas suites de frontend y de firma de backend: no existe un plan de pruebas, runbook de despliegue ni checklist de QA para el comportamiento de extremo a extremo del sistema.
7. **Documentación de estado desactualizada en una integración crítica:** `ORIGINAL_AUTH_INTEGRATION.md` declara la integración como "pendiente de implementación" cuando en realidad está desplegada y en uso activo.
8. **Condición de carrera conocida y no cubierta por pruebas** en el endpoint de cobro de Original Pay, reconocida en la propia documentación de la integración como pendiente de resolución.

Riesgos e Impactos

- El riesgo del punto 1 es de **impacto en integridad transaccional, parcialmente mitigado:** la autenticación entre servicios ya está protegida por pruebas automatizadas, pero la validación de reglas de negocio propias de cada función (por ejemplo, qué campos son obligatorios en un registro) sigue sin ninguna red de seguridad automatizada.
- El riesgo del punto 2 es de **impacto crítico en integridad financiera:** las funciones que efectivamente descuentan o abonan HeartCoin (dentro de las funciones RPC de Postgres, no en las Edge Functions) continúan sin ninguna prueba automatizada que detecte una regresión antes de que impacte el saldo real de un usuario.
- El riesgo del punto 3 es de **impacto en confianza de despliegue, ya parcialmente mitigado:** los flujos cubiertos por la nueva suite de frontend (entrada a la aplicación) tienen ahora una verificación automática real, pero el resto de la aplicación —la mayor parte de su superficie funcional— continúa dependiendo enteramente de pruebas manuales.

- El riesgo del punto 4 es de **impacto operativo y de seguridad de datos**, ya señalado en el documento técnico de despliegue independiente: cualquier verificación manual, hoy, ocurre necesariamente sobre el mismo proyecto que sirve a usuarios reales.
- El riesgo del punto 5 es de **impacto en integridad del esquema de base de datos**, materializable si una migración se aplica fuera de orden o se omite por error humano.
- El riesgo del punto 6 es de **impacto en continuidad del conocimiento**: la verificación real del comportamiento de extremo a extremo depende de que alguien recuerde o localice manualmente los pasajes dispersos donde quedó documentada.
- El riesgo del punto 7 es de **impacto en toma de decisiones**: cualquier persona que use ese documento para decidir si retomar o priorizar trabajo sobre Original Auth partiría de una premisa falsa.
- El riesgo del punto 8 es de **impacto financiero directo, aunque de probabilidad baja**: una condición de carrera en el cobro de HeartCoin podría, en un escenario de solicitudes simultáneas, permitir un cobro por encima del saldo disponible del usuario.

Fortalezas Identificadas

- **Primera cobertura automatizada real tanto en frontend como en backend**: ocho pruebas de frontend (flujo de entrada de la aplicación) y diecinueve pruebas de backend (mecanismo de firma y verificación HMAC de las dos integraciones externas), todas ejecutadas y confirmadas como exitosas — un cambio cualitativo respecto al estado previo, en que ni el frontend ni el backend tenían ninguna prueba automatizada funcional.
- **Priorización correcta de la superficie de mayor riesgo en el backend**: de las múltiples piezas posibles a cubrir, se eligió deliberadamente el mecanismo de autenticación entre servicios —la base de toda la seguridad transaccional del sistema— antes que áreas de menor sensibilidad.
- **Disciplina explícita de no modificar código de producción para lograr testabilidad**: se documentó con precisión por qué la lógica de las siete Edge Functions no pudo cubrirse sin refactorizarlas primero, en vez de forzar una cobertura superficial o inexacta.
- **Diseño de aislamiento correcto y replicable en ambas plataformas**: tanto las pruebas de frontend como las de backend evitan deliberadamente cualquier dependencia de red real, credenciales reales o el único proyecto de Supabase existente, haciendo ambas suites ejecutables en cualquier entorno futuro de integración continua sin modificación adicional.
- **Control de versiones de esquema ya iniciado**: aunque manual, la existencia de `SQL/migrations/` numeradas representa una mejora deliberada sobre el estado previo del proyecto.

- **Análisis estático limpio y estable** en el frontend, sin ningún error de compilación ni de tipo.
- **Testing manual de punta a punta ya realizado y documentado para el flujo transaccional más sensible** (cobro y reembolso automático de Original Pay), incluyendo la verificación explícita de que un webhook reintentado no duplica un reembolso.
- **Reconocimiento explícito y honesto de limitaciones conocidas** dentro de la propia documentación técnica del proyecto (la condición de carrera de `redeem`, la debilidad del webhook de Original Pay, y el alcance real —no exagerado— de la cobertura de pruebas incorporada en esta ronda).

Áreas de Mejora

- Refactorizar, de forma deliberada y revisada, la lógica de negocio inline de al menos las Edge Functions de mayor sensibilidad (`delete-account`, `originalpay-redeem`) para extraerla a funciones exportadas y testeables, replicando el patrón ya usado en `_shared/`.
- Crear `supabase/config.toml` y evaluar levantar una instancia local de Supabase, como paso previo indispensable para poder escribir pruebas automatizadas de las funciones RPC transaccionales.
- Extender la suite de pruebas de frontend más allá del flujo de entrada, priorizando el formulario de registro, el escáner de canje, y los formularios de creación de beneficios/servicios/iniciativas.
- Incorporar un cliente Supabase simulado (*mock*) en las pruebas de frontend, de modo que puedan verificarse también los caminos exitosos de login y registro sin depender de una conexión real al proyecto de producción.
- Evaluar la creación de un segundo proyecto de Supabase dedicado a pruebas, para dejar de depender del único proyecto de producción como entorno de verificación manual.
- Actualizar el encabezado de estado de `ORIGINAL_AUTH_INTEGRATION.md` para reflejar que la integración está implementada y en uso.
- Resolver o, al menos, acotar mediante un bloqueo a nivel de base de datos la condición de carrera conocida en el endpoint de cobro de Original Pay.

Recomendaciones

1. Adoptar ambas suites de pruebas recién incorporadas —frontend y backend— como punto de partida obligatorio: toda pantalla nueva de flujo crítico y todo módulo compartido nuevo de backend debería acompañarse de pruebas equivalentes antes de integrarse, replicando los patrones de aislamiento ya validados.

2. Priorizar la creación de `supabase/config.toml` como siguiente paso concreto de infraestructura de pruebas, dado que es el bloqueador directo para poder cubrir las funciones RPC transaccionales —la pieza de mayor riesgo financiero aún sin ninguna prueba automatizada.
3. Evaluar el refactor de al menos una Edge Function completa (sugerido: `originalpay-redeem`, por ser la de mayor sensibilidad financiera) como caso piloto de extracción de lógica testeable, antes de decidir si se aplica al resto.
4. Corregir de inmediato el encabezado de estado de `ORIGINAL_AUTH_INTEGRATION.md`, dado que es un cambio de bajo esfuerzo con impacto directo en evitar decisiones basadas en información desactualizada.
5. Mantener, como práctica ya demostrada en este proyecto, la documentación honesta de limitaciones conocidas y del alcance real —no exagerado— de cada nueva pieza de cobertura de pruebas que se incorpore.

Conclusión

El despliegue de HeartCoin dio, en esta ronda de trabajo, un paso concreto y verificado en ambos lados del sistema. El frontend pasó de tener un único artefacto de prueba que fallaba al ejecutarse a contar con ocho pruebas reales sobre su flujo de entrada. El backend, que hasta antes de esta ronda no tenía ninguna prueba automatizada en absoluto, incorporó diecinueve pruebas sobre el mecanismo de firma y verificación HMAC que protege sus dos integraciones externas de mayor relevancia transaccional —la superficie de seguridad más crítica del sistema—, instalando además el ejecutor de pruebas de Deno que no estaba disponible en el entorno de desarrollo. Ambas suites comparten un mismo principio de diseño: ninguna depende de red real, credenciales reales, ni del único proyecto de Supabase existente, lo cual las hace directamente reutilizables en un futuro pipeline de integración continua sin modificación adicional. Lo que persiste sin cobertura automatizada, y así debe entenderse con claridad, es la lógica de negocio inline de las siete Edge Functions, la totalidad de las funciones RPC y políticas de RLS de la base de datos —bloqueadas hoy por la ausencia de una instancia de Supabase reproducible localmente—, y la mayor parte de la superficie funcional del frontend posterior al inicio de sesión. El riesgo de mayor impacto potencial que persiste tras esta ronda ya no es la ausencia total de pruebas, sino la concentración de lo que falta: precisamente las funciones que mueven HeartCoin real dentro de la base de datos, que deberían ser la siguiente prioridad de cobertura una vez resuelta la infraestructura de pruebas local necesaria para alcanzarlas.